SENTINEL

Worker threads, a shared queue and the track picture

Prelab 3

Status and deadline

This prelab is **required**, and it has **no marks**. Submit it on Brightspace by **Wednesday, November 18, 2026, by midnight**. The Lab 3 session is on Friday, November 20, 2026.

The oral exam of Lab 3[1] can ask you each question of this prelab. The deliverable of Lab 3 is the code that these questions describe: the Lab 3 Brief[2] states it. The course marks the deliverable out of 80, and it gives 4% of the course grade. The course marks the oral exam out of 20, and it gives 2%. Together they give 6%.

Instructions

In Lab 3, the centre is one process with many threads. The main thread puts each report into a shared queue, and a pool of worker threads takes the reports out and applies them to the track picture, the shared record of what is out there. Nothing protects the shared data in Lab 3. Each question covers one part of that work.

- Do this prelab alone, and write the answers in your own words.
- Each question names the manual pages that you need. If they are not on your machine, install them with `sudo apt install manpages manpages-dev`.
- You do not need the code of Lab 3. Each other fact that you need is in the question.
- You can run a program to check an answer. Give your answer first. Compile a program with threads with `gcc -pthread`.

Q1. What the threads of one process share

Read `man 7 pthreads`, `man 3 pthread_create`, `man 3 pthread_join` and `man 2 fork`. The centre of Lab 3 is one process with a main thread and 8 worker threads.

(a) For each item of this table, write *shared* or *its own*: is it shared by the threads of one process, or does each thread have its own? Then do the same for a parent and its child after `fork()`, as in Lab 2: write *shared*, *a copy* (the child gets a copy of the value of the parent) or *its own* (the child gets a new value).

[1] The oral exam of Lab 3 is in the Lab 4 session. The Submission Guide, Section 3, `docs/submit/submit.pdf`, describes its format.

[2] The Lab 3 Brief, `docs/lab3/lab3_brief.pdf` in your repository. It states the contract of Lab 3, its ideas and its hand-in.

Item	Threads of one process	Parent and child after `fork()`
A global variable, such as `long total;`		
A local variable of a function		
The heap, the memory of `malloc()`		
The open file descriptors		
The process ID		

(b) A worker thread returns from its start function, and no thread calls `pthread_join()` for it. What does the system keep for that thread? Why must the stop of the pool call `pthread_join()` for each worker before it frees the memory of the pool?

Q2. The argument of a thread

Read the description of `arg` in `man 3 pthread_create`. Look at this program:

```
static void *show(void *arg)
{
    int n = *(int *)arg;
    printf("worker %d\n", n);
    return NULL;
}

int main(void)
{
    pthread_t t[4];
    for (int i = 0; i < 4; i++) {
        pthread_create(&t[i], NULL, show, &i);
    }
    for (int k = 0; k < 4; k++) {
        pthread_join(t[k], NULL);
    }
    return 0;
}
```

The main thread can finish its loop before a new thread runs.

(a) The program is meant to print `worker 0` to `worker 3`, one time each. Why can it print another set of numbers, such as `worker 2` two times? Which numbers can a thread print? Why is the result not the same in each run?

(b) Change the program so that each thread prints its own number. The pool of Lab 3 gives each worker the address of its own block, `&workers[i]`. How does that do the same?

Q3. One line of C, three steps

Two threads share `long total = 0;`. Each thread runs `total++;` 1000 times, with no lock. The processor does `total++` in three steps: it LOADS `total` into a register, it ADDS 1 to the register,

and it STORES the register into `total`. Each thread has its own registers. The kernel can stop a thread between any two steps, for example when the time of the thread on the processor ends, and run another thread. A thread runs for some milliseconds before the kernel stops it, and the three steps take some nanoseconds. The first paragraph of `man 7 pthreads` says what the two threads share.

(a) `total` is 5. Write an order of the six steps of the two threads (each one does one `total++`) after which `total` is 6, not 7. Use a table with one line for each step: the thread, the step, the value of its register (a dash before its LOAD), and the value of `total`.

(b) The program runs on ONE processor, so only one thread runs at a time. Can the fault of (a) still occur? What must the kernel do, and when, for it to occur? Is it more frequent or less frequent than on 8 processors? Why does a run that gives the right total prove nothing?

Q4. A ring buffer of 4 slots

The queue of Lab 3 is a ring buffer. It has `capacity` slots, and three numbers: `head`, the slot of the next report to take out; `tail`, the next free slot; and `count`, the number of reports that wait. At the start, all three are 0.

- A push: when `count` is equal to `capacity`, the push is refused. If not, the report goes into slot `tail`, then `tail = (tail + 1) % capacity`, and `count` increases by 1.
- A pop: when `count` is 0, the pop gives nothing. If not, it gives the report in slot `head`, then `head = (head + 1) % capacity`, and `count` decreases by 1.

(a) The capacity is 4. Do these nine operations in order: push A, push B, push C, pop, push D, push E, push F, pop, pop. Give `head`, `tail` and `count` after each operation. Which push is refused? Which report does each pop give?

(b) `head` and `tail` can be equal when the queue is empty, and also when it is full. For each case, give one point of (a), or the start. Why does the queue keep `count`?

Q5. Eight workers on one processor

Read the description of `CLOCK_THREAD_CPUTIME_ID` in `man 2 clock_gettime`. 1000 reports wait in the queue. Each report costs a worker 200 μs of processor time, and nothing else: no wait for a disk or a network. The centre runs on ONE processor, and no other program runs. Ignore the cost of a switch from one thread to another, and suppose that the workers share the reports equally.

(a) What is the shortest time in which 1 worker applies the 1000 reports? What is it for 8 workers? What is it for 8 workers on 8 processors? Explain each number.

(b) For each worker, the centre measures the processor time that the worker used, and divides it by the elapsed time of the run, from `CLOCK_MONOTONIC`. The processor time of a thread does not increase while the thread waits for the processor. That is the share of a processor that the worker got. On one processor, the sum of the shares of the 8 workers is 1.00 or less. Why? Two words name two kinds of work. *Concurrency*: many tasks are in progress at the same time, and they take turns on the processors. *Parallelism*: many tasks run at the same instant, on different processors. Which one does the centre of Lab 3 have, and which number of (a) shows it?